Main menu

Search

Appearance
Create account
Log in

Personal tools

Toggle the table of contents
DOM event

Article
Talk
Read
Edit
View history

Tools
From Wikipedia, the free encyclopedia

This article has multiple issues. Please help improve it or discuss these issues on the talk page. (Learn how and when to remove these messages)
This article is written like a manual or guide. (July 2018)
This article may be too technical for most readers to understand. (July 2018)
DOM (Document Object Model) Events are a signal that something has occurred, or is occurring, and can be triggered by user interactions or by the browser.[1] Client–side scripting languages like JavaScript, JScript, VBScript, and Java can register various event handlers or listeners on the element nodes inside a DOM tree, such as in HTML, XHTML, XUL, and SVG documents.

Examples of DOM Events:

When a user clicks the mouse
When a web page has loaded
When an image has been loaded
When the mouse moves over an element
When an input field is changed
When an HTML form is submitted
When a user presses a key[2]
Historically, like DOM, the event models used by various web browsers had some significant differences which caused compatibility problems. To combat this, the event model was standardized by the World Wide Web Consortium (W3C) in DOM Level 2.

Events
HTML events
Common events
There is a huge collection of events that can be generated by most element nodes:

Mouse events.[3][4]
Keyboard events.
HTML frame/object events.
HTML form events.
User interface events.

Mutation events (notification of any changes to the structure of a document).
Progress events[5] (used by XMLHttpRequest and File API[6]).
Note that the event classification above is not exactly the same as W3C's classification.

| Category | Type | Attribute | Description | Bubbles | Cancelable |
|---|---|---|---|---|---|
| Mouse | click | onclick | Fires when the pointing device button is clicked over an element. A click is defined as a mousedown and mouseup over the same screen location. The sequence of these events is:<br>mousedown<br>mouseup<br>click | Yes | Yes |
| | dblclick | ondblclick | Fires when the pointing device button is double–clicked over an element | Yes | Yes |
| | mousedown | onmousedown | Fires when the pointing device button is pressed over an element | Yes | Yes |
| | mouseup | onmouseup | Fires when the pointing device button is released over an element | Yes | Yes |
| | mouseover | onmouseover | Fires when the pointing device is moved onto an element | Yes | Yes |
| | mousemove[7] | onmousemove | Fires when the pointing device is moved while it is over an element | Yes | Yes |
| | mouseout | onmouseout | Fires when the pointing device is moved away from an element | Yes | Yes |
| | dragstart | ondragstart | Fired on an element when a drag is started. | Yes | Yes |
| | drag | ondrag | This event is fired at the source of the drag, that is, the element where dragstart was fired, during the drag operation. | Yes | Yes |
| | dragenter | ondragenter | Fired when the mouse is first moved over an element while a drag is occurring. | Yes | Yes |
| | dragleave | ondragleave | This event is fired when the mouse leaves an element while a drag is occurring. | Yes | No |
| | dragover | ondragover | This event is fired as the mouse is moved over an element when a drag is occurring. | Yes | Yes |
| | drop | ondrop | The drop event is fired on the element where the drop occurs at the end of the drag operation. | Yes | Yes |
| | dragend | ondragend | The source of the drag will receive a dragend event when the drag operation is complete, whether it was successful or not. | Yes | No |
| Keyboard | keydown | onkeydown | Fires before keypress, when a key on the keyboard is pressed. | Yes | Yes |
| | keypress | onkeypress | Fires after keydown, when a key on the keyboard is pressed. | Yes | Yes |
| | keyup | onkeyup | Fires when a key on the keyboard is released | Yes | Yes |
| HTML frame/object | load | onload | Fires when the user agent finishes loading all content within a document, including window, frames, objects and images<br>For elements, it fires when the target element and all of its content has finished loading | No | No |
| | unload | onunload | Fires when the user agent removes all content from a window or frame<br>For elements, it fires when the target element or any of its content has been removed | No | No |
| | abort | onabort | Fires when an object/image is stopped from loading before completely loaded | Yes | No |
| | error | onerror | Fires when an object/image/frame cannot be loaded properly | Yes | No |
| | resize | onresize | Fires when a document view is resized | Yes | No |
| | scroll | onscroll | Fires when an element or document view is scrolled | No, except | |

that a scroll event on document must bubble to the window[8] No
HTML form select onselect Fires when a user selects some text in a text field, including input and textarea Yes No
change onchange Fires when a control loses the input focus and its value has been modified since gaining focus Yes No
submit onsubmit Fires when a form is submitted Yes Yes
reset onreset Fires when a form is reset Yes No
focus onfocus Fires when an element receives focus either via the pointing device or by tab navigation No No
blur onblur Fires when an element loses focus either via the pointing device or by tabbing navigation No No
User interface focusin (none) Similar to HTML focus event, but can be applied to any focusable element Yes No
focusout (none) Similar to HTML blur event, but can be applied to any focusable element Yes No
DOMActivate (none) Similar to XUL command event. Fires when an element is activated, for instance, through a mouse click or a keypress. Yes Yes
Mutation DOMSubtreeModified (none) Fires when the subtree is modified Yes No
DOMNodeInserted (none) Fires when a node has been added as a child of another node Yes No
DOMNodeRemoved (none) Fires when a node has been removed from a DOM–tree Yes No
DOMNodeRemovedFromDocument (none) Fires when a node is being removed from a document No No
DOMNodeInsertedIntoDocument (none) Fires when a node is being inserted into a document No No
DOMAttrModified (none) Fires when an attribute has been modified Yes No
DOMCharacterDataModified (none) Fires when the character data has been modified Yes No
Progress loadstart (none) Progress has begun. No No
progress (none) In progress. After loadstart has been dispatched. No No
error (none) Progression failed. After the last progress has been dispatched, or after loadstart has been dispatched if progress has not been dispatched. No No
abort (none) Progression is terminated. After the last progress has been dispatched, or after loadstart has been dispatched if progress has not been dispatched. No No
load (none) Progression is successful. After the last progress has been dispatched, or after loadstart has been dispatched if progress has not been dispatched. No No
loadend(none) Progress has stopped. After one of error, abort, or load has been dispatched. No No
Note that the events whose names start with "DOM" are currently not well supported, and for this and other performance reasons are deprecated by the W3C in DOM Level 3. Mozilla and Opera support DOMAttrModified, DOMNodeInserted, DOMNodeRemoved and DOMCharacterDataModified. Chrome and Safari support these events, except for DOMAttrModified.

Touch events
Web browsers running on touch–enabled devices, such as Apple's iOS and Google's Android, generate additional events.[9]:鈥婷 5.3 鈥

Category Type Attribute Description Bubbles Cancelable
Touch touchstart Fires when a finger is placed on the touch surface/screen. Yes Yes
touchend Fires when a finger is removed from the touch surface/screen. Yes Yes
touchmove Fires when a finger already placed on the screen is moved across the screen. Yes Yes

touchenter Fires when a touch point moves onto the interactive area defined by a DOM element.Yes Yes
touchleave Fires when a touch point moves off the interactive area defined by a DOM element. Yes Yes
touchcancel A user agent must dispatch this event type to indicate when a TouchPoint has been disrupted in an implementation−specific manner, such as by moving outside the bounds of the UA window. A user agent may also dispatch this event type when the user places more touch points (The coordinate point at which a pointer (e.g. finger or stylus) intersects the target surface of an interface) on the touch surface than the device or implementation is configured to store, in which case the earliest TouchPoint object in the TouchList should be removed.[9]:鈥婷孌5.9 鈥 Yes No
In the W3C draft recommendation, a TouchEvent delivers a TouchList of Touch locations, the modifier keys that were active, a TouchList of Touch locations within the targeted DOM element, and a TouchList of Touch locations that have changed since the previous TouchEvent.[9]

Apple didn't join this working group, and delayed W3C recommendation of its Touch Events Specification by disclosing patents late in the recommendation process.[10]

Pointer events
Web browsers on devices with various types of input devices including mouse, touch panel, and pen may generate integrated input events. Users can see what type of input device is pressed, what button is pressed on that device, and how strongly the button is pressed when it comes to a stylus pen. As of October 2013, this event is only supported by Internet Explorer 10 and 11.[11]

Category Type Attribute Description Bubbles Cancelable
Pointer pointerdown onpointerdown Fires when the pointing device button is activated, or pressed over an element. Yes Yes
pointerup onpointerup Fires when the pointing device button is released over an element Yes Yes
pointercancel onpointercancel Fires when a pointing device is unlikely to continue to produce event because, for example, the device is used for panning/zooming after a pointerdown event. Yes Yes
pointermove onpointermove Fires when the pointing device is moved while it is over an element Yes Yes
pointerover onpointerover Fires when the pointing device is moved onto an element Yes Yes
pointerout onpointerout Fires when the pointing device is moved away from an element. Also fires after pointerup by pointing device without hovering, or after Yes Yes
pointerenter onpointerenter Fires when the pointing device is moved onto an element, or when the button of the pointing device which does not support hovering is pressed on one of its descendant elements. No Yes
pointerleave onpointerleave Fires when the pointing device is moved away from an element, or when the button of the pointing device which does not support hovering is released over its descendant elements. No Yes
gotpointercapture ongotpointercapture Fires when the pointer is captured by setPointerCapture method. Yes No
lostpointercapture onlostpointercapture Fires when the pointer is released by releasePointerCapture method. Yes No
Indie UI events
Not yet really implemented, the Indie UI working groups want to help web application developers to be able to support standard user interaction events without having to handle different platform specific technical events that could match with it.[12]

Scripting usable interfaces can be difficult, especially when one considers that user interface design patterns differ across software platforms, hardware, and locales, and that

those interactions can be further customized based on personal preference. Individuals are accustomed to the way the interface works on their own system, and their preferred interface frequently differs from that of the web application author's preferred interface.

For example, web application authors, wishing to intercept a user's intent to undo the last action, need to "listen" for all the following events:

Control+Z on Windows and Linux.
Command+Z on Mac OS X.
Shake events on some mobile devices.
It would be simpler to listen for a single, normalized request to "undo" the previous action.

| Category | Type | Description | Bubbles | Cancelable |
|---|---|---|---|---|
| Request | undorequest | Indicates the user desires to "undo" the previous action. (May be superseded by the UndoManager interface.) | Yes | Yes |
| | redorequest | Indicates the user desires to "redo" the previously "undone" action. (May be superseded by the UndoManager interface.) | Yes | No |
| | expandrequest | Indicates the user desires to reveal information in a collapsed section (e.g. a disclosure widget) or branch node in a hierarchy (e.g., a tree view). | Yes | Yes |
| | collapserequest | Indicates the user desires to hide or collapse information in an expanded section (e.g. a disclosure widget) or branch node in a hierarchy (e.g., a tree view). | Yes | Yes |
| | dismissrequest | Indicates the user desires "dismiss" the current view (e.g. canceling a dialog, or closing a popup menu). | Yes | Yes |
| | deleterequest | Indicates the user wants to initiate a "delete" action on the marked element or current view. | Yes | Yes |
| Focus Request | directionalfocusrequest | Initiated when the user agent sends a "direction focus" request to the web application. Web authors should not use or register for directionalfocusrequest events when standard browser focus and blur events are sufficient. Using these events unnecessarily could result in reduced performance or negative user experience. | Yes | Yes |
| | linearfocusrequest | Initiated when the user agent sends a "linear focus" request to the web application. Web authors should not use or register for linearfocusrequest events when standard browser focus and blur events are sufficient. This event type is only necessary on specialized control types such as data grids where the logical next element may not be focusable or even in the DOM until requested. Using these events unnecessarily could result in reduced performance or negative user experience. | Yes | Yes |
| | palettefocusrequest | Initiated when the user agent sends a "palette focus" request to the web application. Web app authors receiving this event should move focus to the first palette in the web application, or cycle focus between all available palettes. Note: palettes are sometimes referred to as non–modal dialogs or inspector windows. | Yes | Yes |
| | toolbarfocusrequest | Initiated when the user agent sends a "toolbar focus" request to the web application. Web app authors receiving this event should move focus to the main toolbar in the web application, or cycle focus between all available toolbars. | Yes | Yes |
| Manipulation Request | moverequest | Initiated when the user agent sends a move request to the web application with accompanying x/y delta values. This is used, for example, when moving an object to a new location on a layout canvas. | Yes | Yes |
| | panrequest | Initiated when the user agent sends a pan request to the web application with accompanying x/y delta values. This is used, for example, when changing the center point while panning a map or another custom image viewer. | Yes | Yes |
| | rotationrequest | Initiated when the user agent sends a rotation request to the web application with accompanying origin x/y values and a rotation value in degrees. | Yes | Yes |
| | zoomrequest | Initiated when the user agent sends a zoom request to the web application with accompanying origin x/y values and the zoom scale factor. | Yes | Yes |
| Scroll Request | scrollrequest | Initiated when the user agent sends a scroll request to the web application with accompanying x/y delta values or one of the other defined scrollType values. Authors should only use this event and uiaction with custom scroll views. | Yes | |

Yes
ValueChange Request valuechangerequest Initiated when the user agent sends a value change request to the web application. Used on custom range controls like sliders, carousels, etc. Yes Yes
Internet Explorer–specific events
In addition to the common (W3C) events, two major types of events are added by Internet Explorer. Some of the events have been implemented as de facto standards by other browsers.

Clipboard events.
Data binding events.[clarification needed]
Category Type Attribute Description Bubbles Cancelable
Clipboard cut oncut Fires after a selection is cut to the clipboard. Yes Yes
copy oncopy Fires after a selection is copied to the clipboard.Yes Yes
paste onpaste Fires after a selection is pasted from the clipboard. Yes Yes
beforecut onbeforecut Fires before a selection is cut to the clipboard. Yes Yes
beforecopy onbeforecopy Fires before a selection is copied to the clipboard. Yes Yes
beforepaste onbeforepaste Fires before a selection is pasted from the clipboard. Yes Yes
Data binding afterupdate onafterupdate Fires immediately after a databound object has been updated. Yes No
beforeupdate onbeforeupdate Fires before a data source is updated. Yes Yes
cellchange oncellchange Fires when a data source has changed. Yes No
dataavailable ondataavailable Fires when new data from a data source become available. Yes No
datasetchanged ondatasetchanged Fires when content at a data source has changed. Yes No
datasetcomplete ondatasetcomplete Fires when transfer of data from the data source has completed. Yes No
errorupdate onerrorupdate Fires if an error occurs while updating a data field. Yes No
rowenter onrowenter Fires when a new row of data from the data source is available. Yes No
rowexit onrowexit Fires when a row of data from the data source has just finished. No Yes
rowsdelete onrowsdelete Fires when a row of data from the data source is deleted. Yes No
rowinserted onrowinserted Fires when a row of data from the data source is inserted. Yes No
Mouse contextmenu oncontextmenu Fires when the context menu is shown. Yes Yes
drag ondrag Fires when during a mouse drag (on the moving Element). Yes Yes
dragstart ondragstart Fires when a mouse drag begins (on the moving Element). Yes Yes
dragenter ondragenter Fires when something is dragged onto an area (on the target Element). Yes Yes
dragover ondragover Fires when a drag is held over an area (on the target Element). Yes Yes
dragleave ondragleave Fires when something is dragged out of an area (on the target Element). Yes Yes
dragend ondragend Fires when a mouse drag ends (on the moving Element). Yes Yes
drop ondrop Fires when a mouse button is released over a valid target during a drag (on the target Element). Yes Yes
selectstart onselectstart Fires when the user starts to select text. Yes Yes
Keyboard help onhelp Fires when the user initiates help. Yes Yes
HTML frame/object beforeunload onbeforeunload Fires before a document is

unloaded. No Yes
stop onstop Fires when the user stops loading the object. (unlike abort, stop event can be attached to document) No No
HTML form beforeeditfocus onbeforeeditfocus Fires before an element gains focus for editing. Yes Yes
Marquee start onstart Fires when a marquee begins a new loop. No No
finish onfinish Fires when marquee looping is complete. No Yes
bounce onbounce Fires when a scrolling marquee bounces back in the other direction. No Yes
Miscellaneous beforeprint onbeforeprint Fires before a document is printed No No
afterprint onafterprint Fires immediately after the document prints. No No
propertychange onpropertychange Fires when the property of an object is changed. No No
filterchange onfilterchange Fires when a filter changes properties or finishes a transition. No No
readystatechange onreadystatechange Fires when the readyState property of an element changes. No No
losecapture onlosecapture Fires when the releaseCapture method is invoked. No No
Note that Mozilla, Safari and Opera also support the readystatechange event for the XMLHttpRequest object. Mozilla also supports the beforeunload event using the traditional event registration method (DOM Level 0). Mozilla and Safari also support contextmenu, but Internet Explorer for Mac does not.

Note that Firefox 6 and later support the beforeprint and afterprint events.

XUL events
In addition to the common (W3C) events, Mozilla defined a set of events that work only with XUL elements.[citation needed]

Category Type Attribute Description Bubbles Cancelable
Mouse DOMMouseScroll DOMMouseScroll Fires when the mouse wheel is moved, causing the content to scroll. Yes Yes
dragdrop ondragdrop Fires when the user releases the mouse button to drop an object being dragged. No No
dragenter ondragenter Fires when the mouse pointer first moves over an element during a drag. It is similar to the mouseover event but occurs while dragging. No No
dragexit ondragexit Fires when the mouse pointer moves away from an element during a drag. It is also called after a drop on an element. It is similar to the mouseout event but occurs during a drag. No No
draggesture ondraggesture Fires when the user starts dragging the element, usually by holding down the mouse button and moving the mouse. No No
dragover ondragover Related to the mousemove event, this event is fired while something is being dragged over an element. No No
Input CheckboxStateChange Fires when a checkbox is checked or unchecked, either by the user or a script. No No
RadioStateChange Fires when a radio button is selected, either by the user or a script. No No
close onclose Fires when a request has been made to close the window. No Yes
command oncommand Similar to W3C DOMActivate event. Fires when an element is activated, for instance, through a mouse click or a keypress. No No
input oninput Fires when a user enters text in a textbox. Yes No
User interface DOMMenuItemActive DOMMenuItemActive Fires when a menu or menuitem is hovered over, or highlighted. Yes No
DOMMenuItemInactive DOMMenuItemInactive Fires when a menu or menuitem is no longer being hovered over, or highlighted. Yes No

contextmenu oncontextmenu Fires when the user requests to open the context menu for the element. The action to do this varies by platform, but it will typically be a right click. No Yes
overflow onoverflow Fires a box or other layout element when there is not enough space to display it at full size. No No
overflowchanged onoverflowchanged Fires when the overflow state changes. No No
underflow onunderflow Fires to an element when there becomes enough space to display it at full size. No No
popuphidden onpopuphidden Fires to a popup after it has been hidden. No No
popuphiding onpopuphiding Fires to a popup when it is about to be hidden. No No
popupshowing onpopupshowing Fires to a popup just before it is popped open. No Yes
popupshown onpopupshown Fires to a popup after it has been opened, much like the onload event is sent to a window when it is opened. No No
Command broadcast onbroadcast Placed on an observer. The broadcast event is sent when the attributes of the broadcaster being listened to are changed. No No
commandupdate oncommandupdate Fires when a command update occurs. No No
Other events
For Mozilla and Opera 9, there are also undocumented events known as DOMContentLoaded and DOMFrameContentLoaded which fire when the DOM content is loaded. These are different from "load" as they fire before the loading of related files (e.g., images). However, DOMContentLoaded has been added to the HTML 5 specification.[13] The DOMContentLoaded event was also implemented in the Webkit rendering engine build 500+.[14][15] This correlates to all versions of Google Chrome and Safari 3.1+. DOMContentLoaded is also implemented in Internet Explorer 9.[16]

Opera 9 also supports the Web Forms 2.0 events DOMControlValueChanged, invalid, forminput and formchange.

Event flow
Consider the situation when two event targets participate in a tree. Both have event listeners registered on the same event type, say "click". When the user clicks on the inner element, there are two possible ways to handle it:

Trigger the elements from outer to inner (event capturing). This model is implemented in Netscape Navigator.
Trigger the elements from inner to outer (event bubbling). This model is implemented in Internet Explorer and other browsers.[17]
W3C takes a middle position in this struggle.[18]:鈥婩鈥1.2 鈥

According to the W3C, events go through three phases when an event target participates in a tree:

The capture phase: the event travels down from the root event target to the target of an event
The target phase: the event travels through the event target
The bubble phase (optional): the event travels back up from the target of an event to the root event target. The bubble phase will only occur for events that bubble (where event.bubbles == true)
You can find a visualization of this event flow at https://domevents.dev

Stopping events
While an event is travelling through event listeners, the event can be stopped with event.stopPropagation() or event.stopImmediatePropagation()

event.stopPropagation(): the event is stopped after all event listeners attached to the current event target in the current event phase are finished
event.stopImmediatePropagation(): the event is stopped immediately and no further event listeners are executed
When an event is stopped it will no longer travel along the event path. Stopping an event does not cancel an event.

Legacy mechanisms to stop an event
Set the event.cancelBubble to true (Internet Explorer)
Set the event.returnValue property to false
Canceling events
A cancelable event can be canceled by calling event.preventDefault(). Canceling an event will opt out of the default browser behaviour for that event. When an event is canceled, the event.defaultPrevented property will be set to true. Canceling an event will not stop the event from traveling along the event path.

Event object
The Event object provides a lot of information about a particular event, including information about target element, key pressed, mouse button pressed, mouse position, etc. Unfortunately, there are very serious browser incompatibilities in this area. Hence only the W3C Event object is discussed in this article.

Event properties
Name Type Description
type DOMString The name of the event (case–insensitive in DOM level 2 but case–sensitive in DOM level 3 [19]).
target EventTarget Used to indicate the EventTarget to which the event was originally dispatched.
currentTarget EventTarget Used to indicate the EventTarget whose EventListeners are currently being processed.
eventPhase unsigned short Used to indicate which phase of event flow is currently being evaluated.
bubblesboolean Used to indicate whether or not an event is a bubbling event.
cancelable boolean Used to indicate whether or not an event can have its default action prevented.
timeStamp DOMTimeStamp Used to specify the time (in milliseconds relative to the epoch) at which the event was created.
Event methods
Name Argument type Argument name Description
stopPropagation To prevent further propagation of an event during event flow.
preventDefault To cancel the event if it is cancelable, meaning that any default action normally taken by the implementation as a result of the event will not occur.
initEvent DOMString eventTypeArg Specifies the event type.
boolean canBubbleArg Specifies whether or not the event can bubble.
boolean cancelableArg Specifies whether or not the event's default action can be prevented.
Event handling models
DOM Level 0
This event handling model was introduced by Netscape Navigator, and remains the most cross–browser model as of 2005.[citation needed] There are two model types: the inline model and the traditional model.

Inline model
In the inline model,[20] event handlers are added as attributes of elements. In the example below, an alert dialog box with the message "Hey Joe" appears after the hyperlink is clicked. The default click action is cancelled by returning false in the event handler.

```
<!doctype html>
<html lang="en">
<head>
	<meta charset="utf-8">
	<title>Inline Event Handling</title>
</head>
<body>

	<h1>Inline Event Handling</h1>

	<p>Hey <a href="http://www.example.com" onclick="triggerAlert('Joe'); return false;">Joe</a>!</p>

	<script>
		function triggerAlert(name) {
			window.alert("Hey " + name);
		}
	</script>
</body>
</html>
```

One common misconception[citation needed] with the inline model is the belief that it allows the registration of event handlers with custom arguments, e.g. name in the triggerAlert function. While it may seem like that is the case in the example above, what is really happening is that the JavaScript engine of the browser creates an anonymous function containing the statements in the onclick attribute. The onclick handler of the element would be bound to the following anonymous function:

```
function () {
	triggerAlert('Joe');
	return false;
}
```

This limitation of the JavaScript event model is usually overcome by assigning attributes to the function object of the event handler or by using closures.

Traditional model

In the traditional model,[21] event handlers can be added or removed by scripts. Like the inline model, each event can only have one event handler registered. The event is added by assigning the handler name to the event property of the element object. To remove an event handler, simply set the property to null:

```
<!doctype html>
<html lang="en">
<head>
	<meta charset="utf-8">
	<title>Traditional Event Handling</title>
</head>

<body>
	<h1>Traditional Event Handling</h1>

	<p>Hey Joe!</p>

	<script>
		var triggerAlert = function () {
			window.alert("Hey Joe");
		};
```

```
                // Assign an event handler
                document.onclick = triggerAlert;

                // Assign another event handler
                window.onload = triggerAlert;

                // Remove the event handler that was just assigned
                window.onload = null;
        </script>
</body>
</html>
```

To add parameters:

```
var name = 'Joe';
document.onclick = (function (name) {
        return function () {
                alert('Hey ' + name + '!');
        };
}(name));
```

Inner functions preserve their scope.

DOM Level 2

The W3C designed a more flexible event handling model in DOM Level 2.[18]

| Name | Description | Argument type | Argument name |
|---|---|---|---|
| addEventListener | Allows the registration of event listeners on the event target. | DOMString | type |
| | | EventListener | listener |
| | | boolean | useCapture |
| removeEventListener | Allows the removal of event listeners from the event target. | DOMString | type |
| | | EventListener | listener |
| | | boolean | useCapture |
| dispatchEvent | Allows sending the event to the subscribed event listeners. | Event | evt |

Some useful things to know 聽:

To prevent an event from bubbling, developers must call the stopPropagation() method of the event object.

To prevent the default action of the event to be called, developers must call the preventDefault() method of the event object.

The main difference from the traditional model is that multiple event handlers can be registered for the same event. The useCapture option can also be used to specify that the handler should be called in the capture phase instead of the bubbling phase. This model is supported by Mozilla, Opera, Safari, Chrome and Konqueror.

A rewrite of the example used in the traditional model

```
<!doctype html>
<html lang="en">
<head>
        <meta charset="utf-8">
        <title>DOM Level 2</title>
</head>

<body>
        <h1>DOM Level 2</h1>
```

```
        <p>Hey Joe!</p>


        <script>
                var heyJoe = function () {
                        window.alert("Hey Joe!");
                }

                // Add an event handler
                document.addEventListener( "click", heyJoe, true );    // capture phase

                // Add another event handler
                window.addEventListener( "load", heyJoe, false );    // bubbling phase

                // Remove the event handler just added
                window.removeEventListener( "load", heyJoe, false );
        </script>

</body>
</html>
```

Internet Explorer–specific model

Microsoft Internet Explorer prior to version 8 does not follow the W3C model, as its own model was created prior to the ratification of the W3C standard. Internet Explorer 9 follows DOM level 3 events,[22] and Internet Explorer 11 deletes its support for Microsoft–specific model.[23]

| Name | Description | Argument type | Argument name |
|---|---|---|---|
| attachEvent | Similar to W3C's addEventListener method. | String | sEvent |
| Pointer | fpNotify | | |
| detachEvent | Similar to W3C's removeEventListener method. | String | sEvent |
| Pointer | fpNotify | | |
| fireEvent | Similar to W3C's dispatchEvent method. | String | sEvent |
| Event | oEventObject | | |

Some useful things to know 聽:

To prevent an event bubbling, developers must set the event's cancelBubble property.
To prevent the default action of the event to be called, developers must set the event's returnValue property.
The this keyword refers to the global window object.
Again, this model differs from the traditional model in that multiple event handlers can be registered for the same event. However the useCapture option can not be used to specify that the handler should be called in the capture phase. This model is supported by Microsoft Internet Explorer and Trident based browsers (e.g. Maxthon, Avant Browser).

A rewrite of the example used in the old Internet Explorer–specific model

```
<!doctype html>
<html lang="en">
<head>
        <meta charset="utf-8">
        <title>Internet Explorer–specific model</title>
</head>
<body>
        <h1>Internet Explorer–specific model</h1>

        <p>Hey Joe!</p>

        <script>
```

```
            var heyJoe = function () {
                window.alert("Hey Joe!");
            }

            // Add an event handler
            document.attachEvent("onclick", heyJoe);

            // Add another event handler
            window.attachEvent("onload", heyJoe);

            // Remove the event handler just added
            window.detachEvent("onload", heyJoe);
        </script>

</body>
</html>
```

References


"DOM Standard". dom.spec.whatwg.org. Retrieved 2021–05–25.

"JavaScript DOM Events". www.w3schools.com. Retrieved 2019–08–03.

"7.8 Drag and drop 鈥 HTML5".

"HTML Drag and Drop API". 28 March 2024.

"Progress Events".

"File API".

"Element: Mousemove event – Web APIs | MDN". 22 December 2023.

"Document Object Model (DOM) Level 3 Events Specification (working draft)". W3C. Retrieved 2013–04–17.

"Touch Events version 2 – W3C Editor's Draft". W3C. 14 November 2011. Retrieved 10 December 2011.

"Apple using patents to undermine open standards again". opera.com. 9 December 2011. Retrieved 9 December 2011.

"Pointer Events".

"IndieUI: Events 1.0".

"HTML Standard".

[1] Archived June 11, 2010, at the Wayback Machine

"Which browsers support native DOMContentLoaded event? 芦 聽 Perfection Labs Development Tips". 29 June 2011. Archived from the original on 29 June 2011.

"Test Drive Redirect". Archived from the original on 2010–05–08. Retrieved 2010–05–06.

"Introduction to Events". Quirksmode.org. Retrieved 15 September 2012.

"Document Object Model (DOM) Level 2 Events Specification". W3C. 13 November 2000. Retrieved 15 September 2012.

"Document Object Model (DOM) Level 3 Events Specification (working draft)". W3C. Retrieved 2013–04–17.

"Early event handlers". Quirksmode.org. Retrieved 15 September 2012.

"Traditional event registration model". Quirksmode.org. Retrieved 15 September 2012.

"DOM Level 3 Events support in IE9". Microsoft. March 26, 2010. Retrieved 2010–03–28.

"Compatibility changes in IE11 Preview". Microsoft. September 9, 2013. Retrieved 2013–10–05.


Further reading


Deitel, Harvey. (2002). Internet and World Wide Web: how to program (Second Edition). ISBN 聽 0–13–030897–8

The Mozilla Organization. (2009). DOM Event Reference. Retrieved August 25, 2009.

Quirksmode (2008). Event compatibility tables. Retrieved November 27, 2008.

http://www.sitepen.com/blog/2008/07/10/touching–and–gesturing–on–the–iphone/


External links

Document Object Model (DOM) Level 2 Events Specification

Document Object Model (DOM) Level 3 Events Working Draft

DOM4: Events (Editor's Draft)

UI Events Working Draft
Pointer Events W3C Candidate Recommendation
MSDN PointerEvent
domevents.dev – A visualizer to learn about DOM Events through exploration
JS fiddle for Event Bubbling and Capturing
vte
Web interfaces
Categories: World Wide Web Consortium standardsApplication programming interfacesEvents (computing)
This page was last edited on 15 May 2024, at 13:11 聽(UTC).